Tensor Cores

Tensor Core Instructions (Nvidia)

- **Warp Level Matrix Multiply-Accumulate Instructions**
  - **D = A * B + C A,B,C,and D are matrices (a tile of large matrices)**
  - The shapes of all three matrix operands are collectively described by the tuple `MxNxK`, where A is an `MxK` matrix, B is a `KxN` matrix, while C and D are `MxN` matrices.

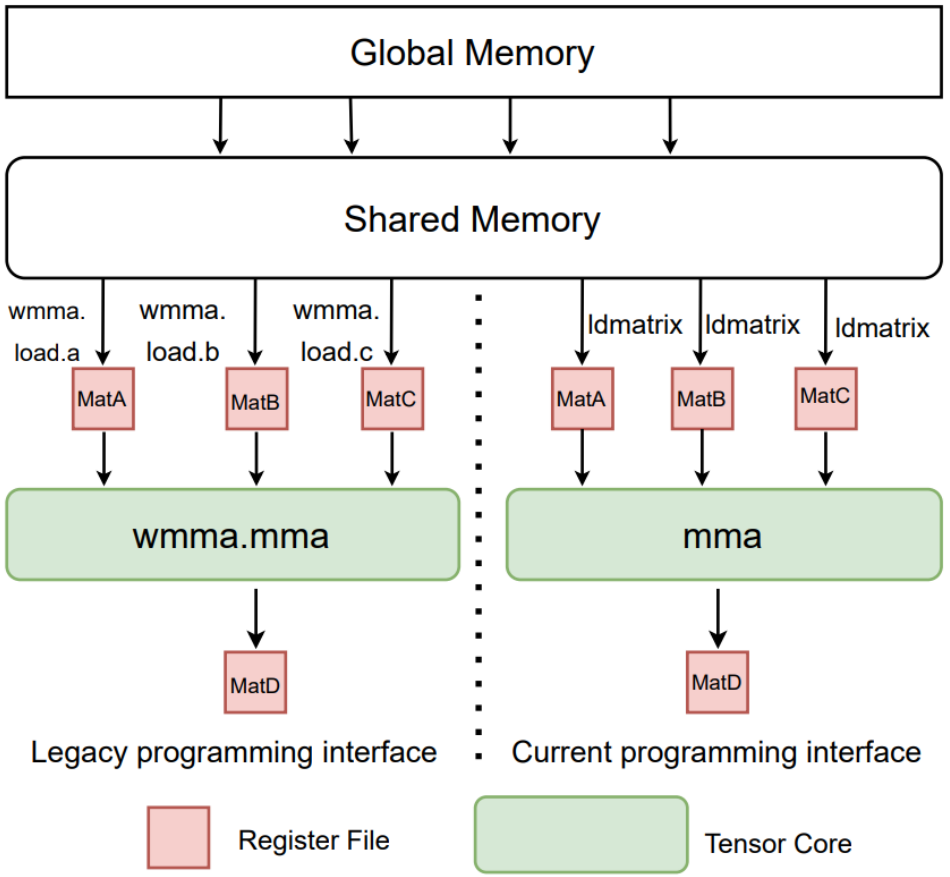
- wmma.load loads A, B, and C based on different M,N,K and precision
- wmma.mma performs A*B + C on tensor cores
- wmma.store stores D to memory based on M, N and precision

Tensor Core Architecture

- Target workloads: D= A*B+C
  - Dense matrices
  - Sparse-dense matrix multiplication. A is sparse, B and C are dense

Tensor Core Programming Model

- Programming Model: At PTX level
  - Per warp: wmma.load.a, wmma.load.b, wmma.load.c, wmma.mma (NVIDIA Volta/Turing/Ampere)
  - Per thread/per warp: ldmatrix, mma (NVIDIA Turing/Ampere)
  - 16x16 MMA



Tensor Core PTX Instructions (Raihan et al. 2019)

```
wmma.load.a.sync.layout.shape.type ra, [pa] {stride};
wmma.load.b.sync.layout.shape.type rb, [pb] {stride};
wmma.load.c.sync.layout.shape.type rc, [pc] {stride};
wmma.mma.sync.alayout.blayout.shape.dtype.ctype rd, ra, rb, rc;
wmma.store.d.sync.layout.shape.type rd, [pd] {stride};
```

Sync: indicates that the instruction waits for all threads in the warp to synchronize before beginning execution
Layout: specifies whether an operand matrix is stored in memory with a row-major or column-major layout.
Shape: represents the fragment size of the operand matrices (e.g., 16 × 16 × 16 is specified by setting shape to m16n16k16).
Type: indicates the precision of the operand matrices

Stride: Typically, input tiles loaded from memory are a portion of a larger matrix. To help accessing tiles of a larger matrix, wmma.load and wmma.store support strided-memory access. The “stride” operand specifies the beginning of each row (or column).

PTX MMA

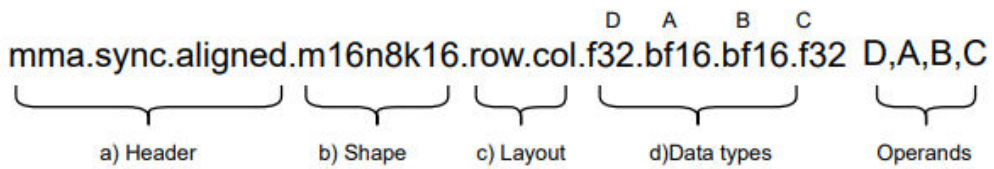


Fig. 5: The mma instruction performs a D = A×B + C MatMul. Segment m16n8k16 defines The shape of A(m×k), B(k×n), C(m×n), and D(m×n). The layout indicates the matrices are either row-major or col-major. The final instruction segment indicates the numeric types of D, A, B, and C respectively.

Code Example

https://bruce-lee-ly.medium.com/nvidia-tensor-core-introduction-to-wmma-api-programming-21bcfee4ec45

Data Movement Instructions

1. Load from shared memory instructions (per thread)
2. Ldmatrix instructions: per warp or per thread. One instruction can be used to load matrix a, b, or c. Data layout more flexible as the 16-byte groups do not need to be consecutive (each 16-byte group is consecutive)
3. Wmma.load: different instructions for matrix a, b, and c. Require continuous matrix data layout. Can load from either global or shared memory (depending on p).

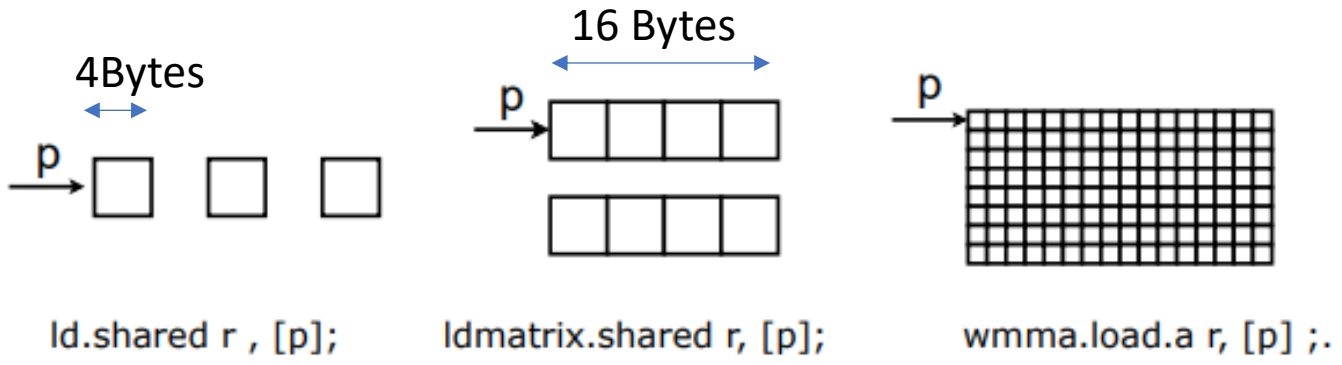


Fig. 12: Differences of three data movement instructions.

Ldmatrix instruction

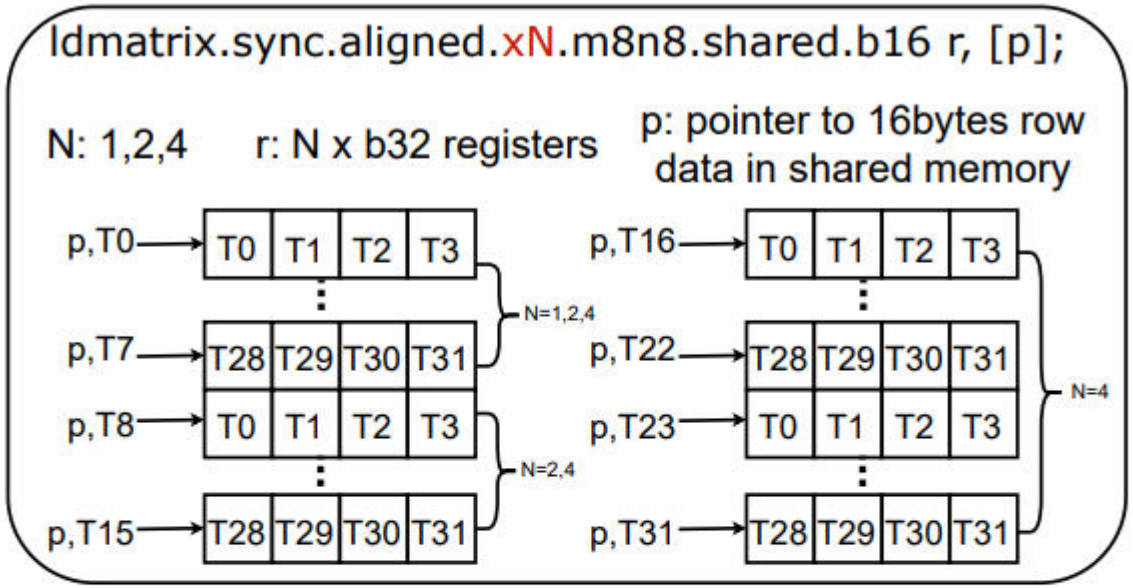


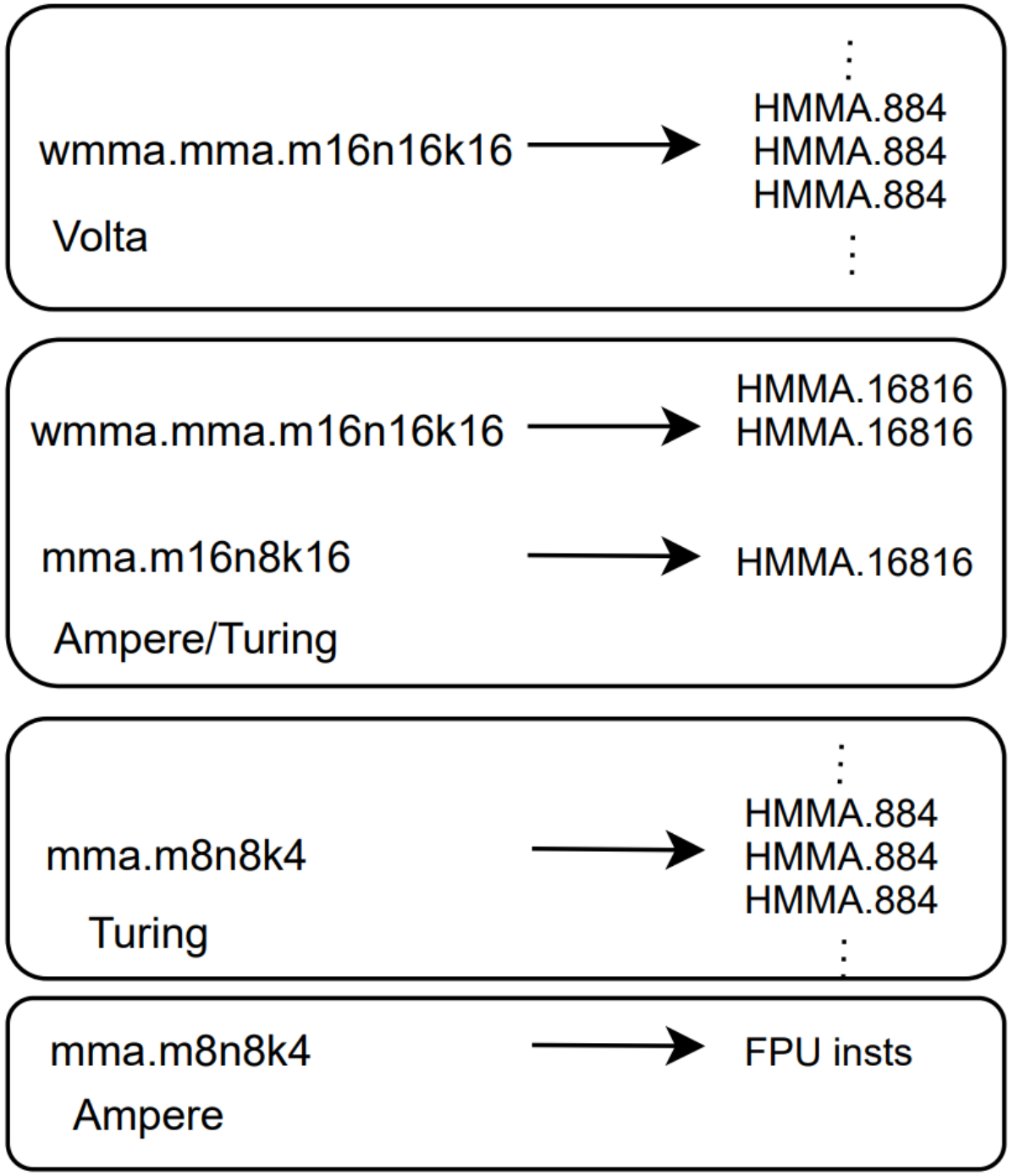
Fig. 13: Functionality overview. p is the source operand and is provided by threads. If N = 4, then every thread will provide a valid p. if N < 4, partial threads (e.g. T0 - T7 for N = 1) will provide a valid p, and other threads' p will be ignored.

When N is 1: first 8 threads will be used to load all the data. One thread loads in a 16-byte group, which is consecutive in shared memory

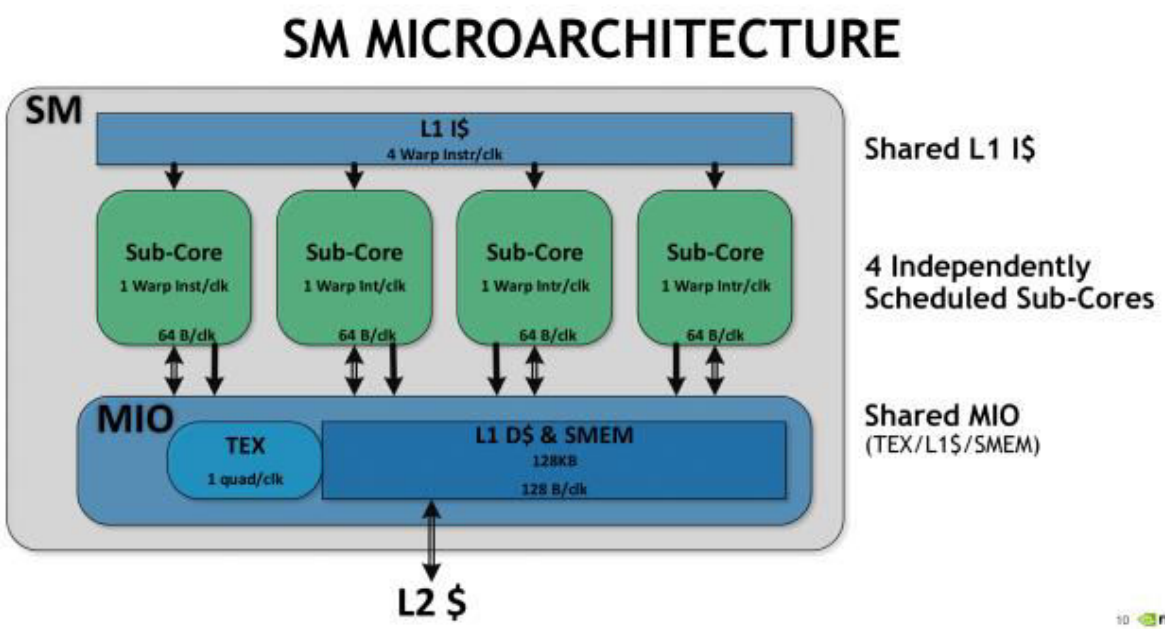
When N is 4: all 32 threads will be used to load the data. One thread loads in a 16-byte group, which is consecutive in shared memory

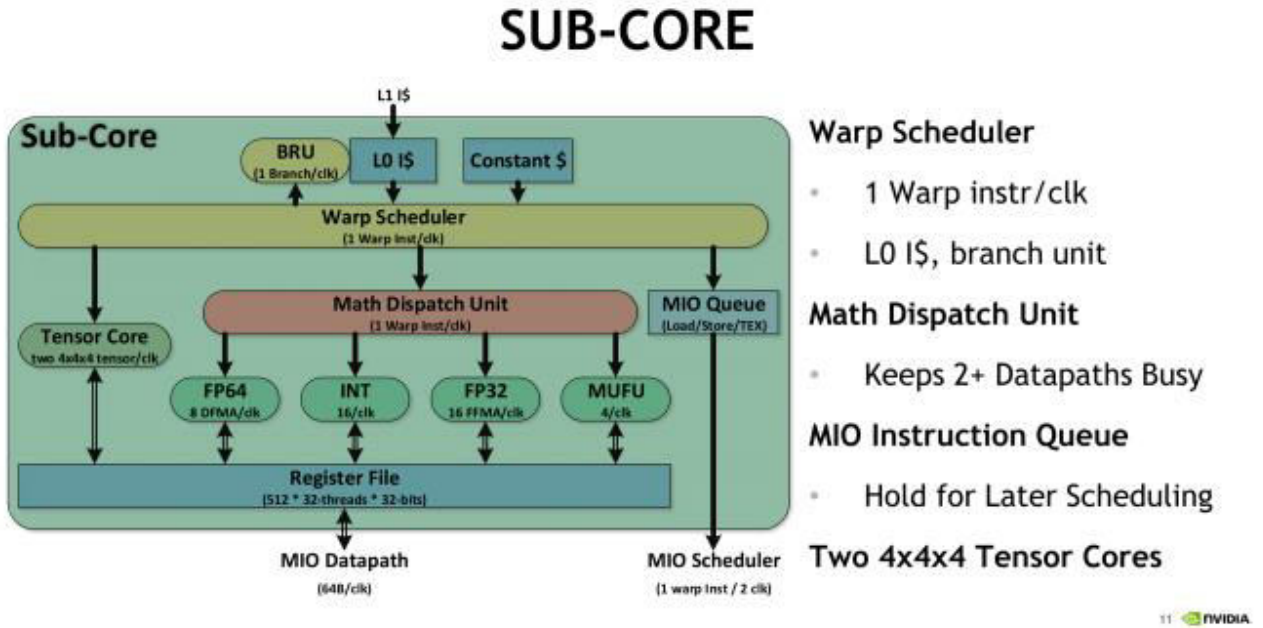
From PTX to SASS

- HMMA SASS instructions

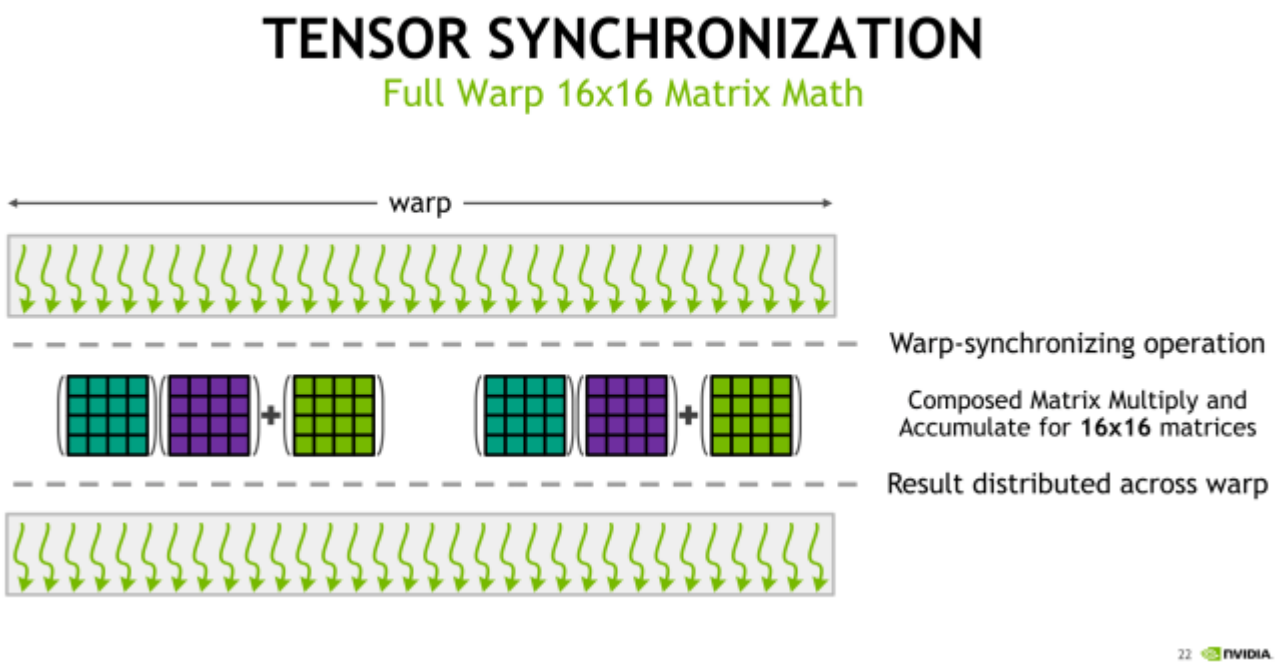


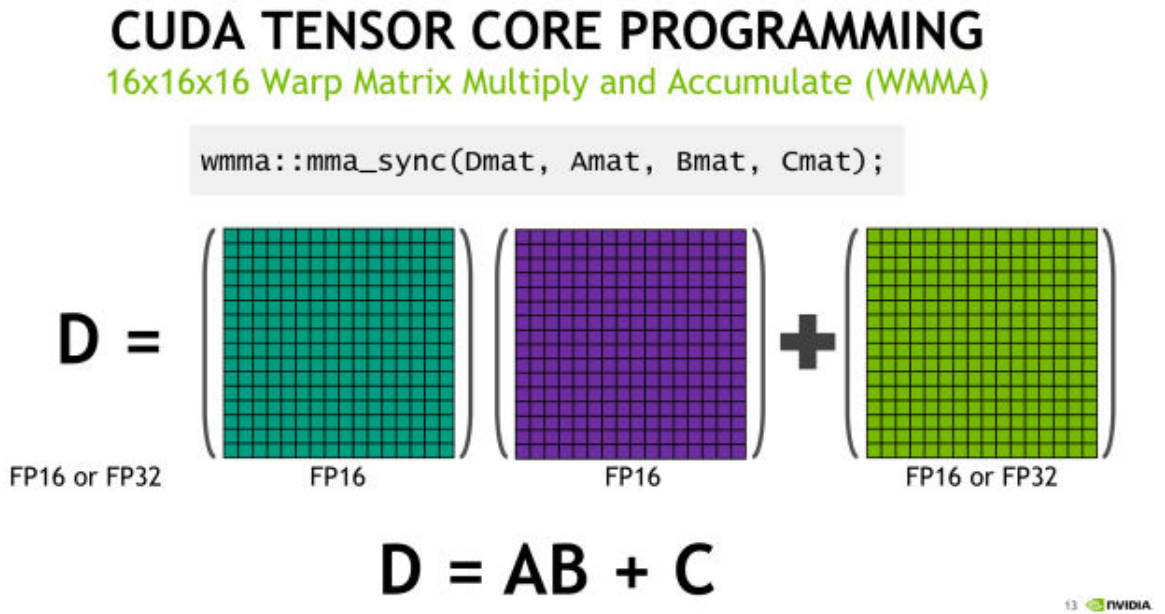
SM Microarchitecture with Tensor Cores



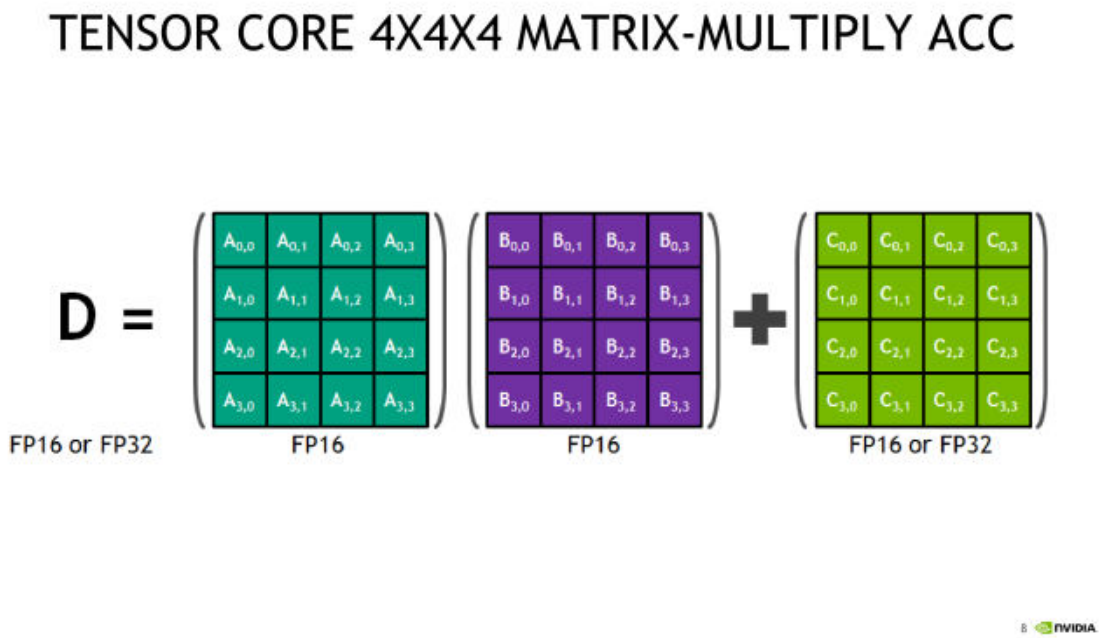


WMMA Instructions (Volta)

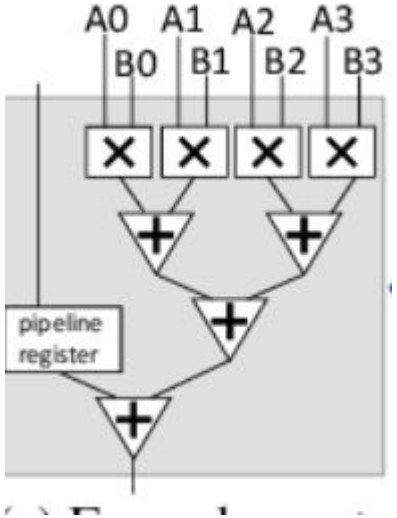




Tensor Core MMA Implementation



Basic functional unit: Four-Element Dot-Product (FEDP)



Implementing WMMA with HMMA instructions

- Conceptually, the tensor cores operate on 4 x 4 submatrices to calculate the larger 16 x 16 matrix, involving Volta's cooperative groups and new scheduling model.
- A warp is broken down into four octets; each octet is broken down into two thread groups (i.e., each thread group contains 4 threads)

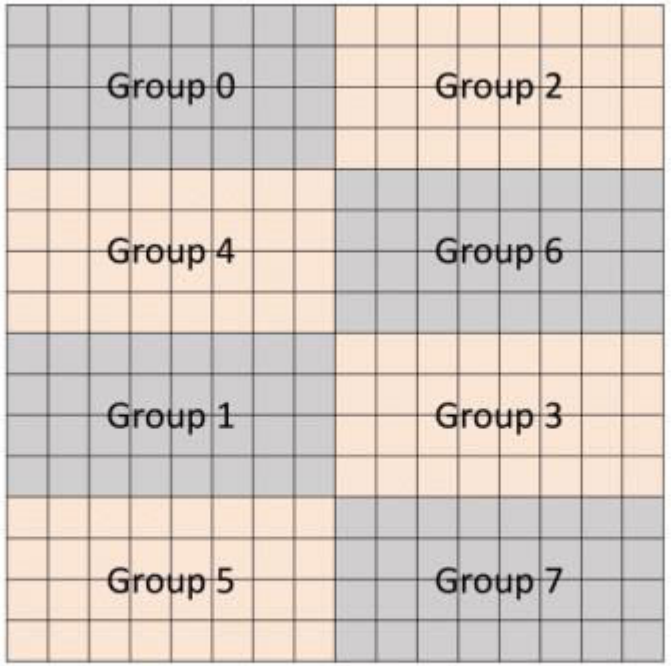
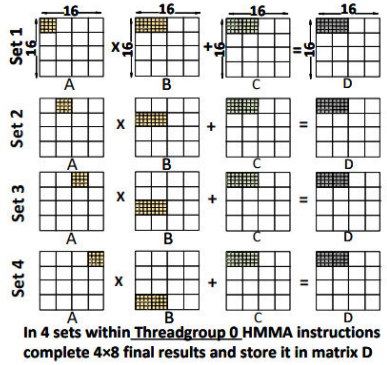
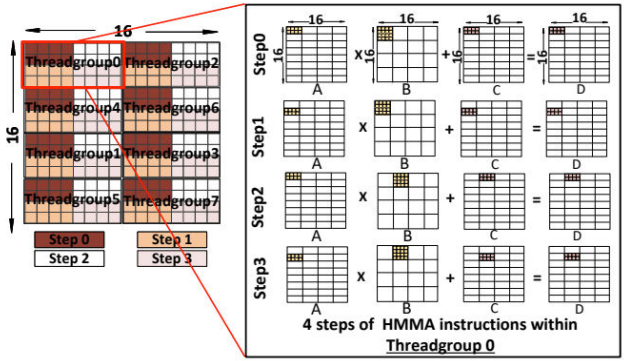


Figure 4.5: Mapping between positions in matrix C and thread group indices.

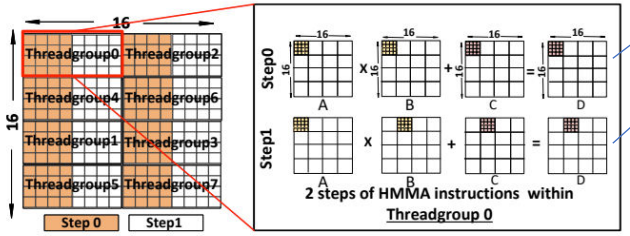
Implementing WMMA with HMMA instructions



(a) Elements accessed in each "Set"



(b) Elements accessed in each "Step" (mixed-precision mode).



(c) Elements accessed in each "Step" (FP16 mode).

		Cumulative Clock Cycles
SET1	HMMA.884.F32.F32.STEP0 R8, R24.reuse.COL, R22.reuse.ROW, R8;	10
	HMMA.884.F32.F32.STEP1 R10, R24.reuse.COL, R22.reuse.ROW, R10;	12
	HMMA.884.F32.F32.STEP2 R4, R24.reuse.COL, R22.reuse.ROW, R4;	14
	HMMA.884.F32.F32.STEP3 R6, R24.COL, R22.ROW, R6;	18
SET2	HMMA.884.F32.F32.STEP0 R8, R20.reuse.COL, R18.reuse.ROW, R8;	20
	HMMA.884.F32.F32.STEP1 R10, R20.reuse.COL, R18.reuse.ROW, R10;	22
	HMMA.884.F32.F32.STEP2 R4, R20.reuse.COL, R18.reuse.ROW, R4;	24
	HMMA.884.F32.F32.STEP3 R6, R20.COL, R18.ROW, R6;	28
SET3	HMMA.884.F32.F32.STEP0 R8, R14.reuse.COL, R12.reuse.ROW, R8;	30
	HMMA.884.F32.F32.STEP1 R10, R14.reuse.COL, R12.reuse.ROW, R10;	32
	HMMA.884.F32.F32.STEP2 R4, R14.reuse.COL, R12.reuse.ROW, R4;	34
	HMMA.884.F32.F32.STEP3 R6, R14.COL, R12.ROW, R6;	38
SET4	HMMA.884.F32.F32.STEP0 R8, R16.reuse.COL, R2.reuse.ROW, R8;	40
	HMMA.884.F32.F32.STEP1 R10, R16.reuse.COL, R2.reuse.ROW, R10;	42
	HMMA.884.F32.F32.STEP2 R4, R16.reuse.COL, R2.reuse.ROW, R4;	44
	HMMA.884.F32.F32.STEP3 R6, R16.COL, R2.ROW, R6;	54

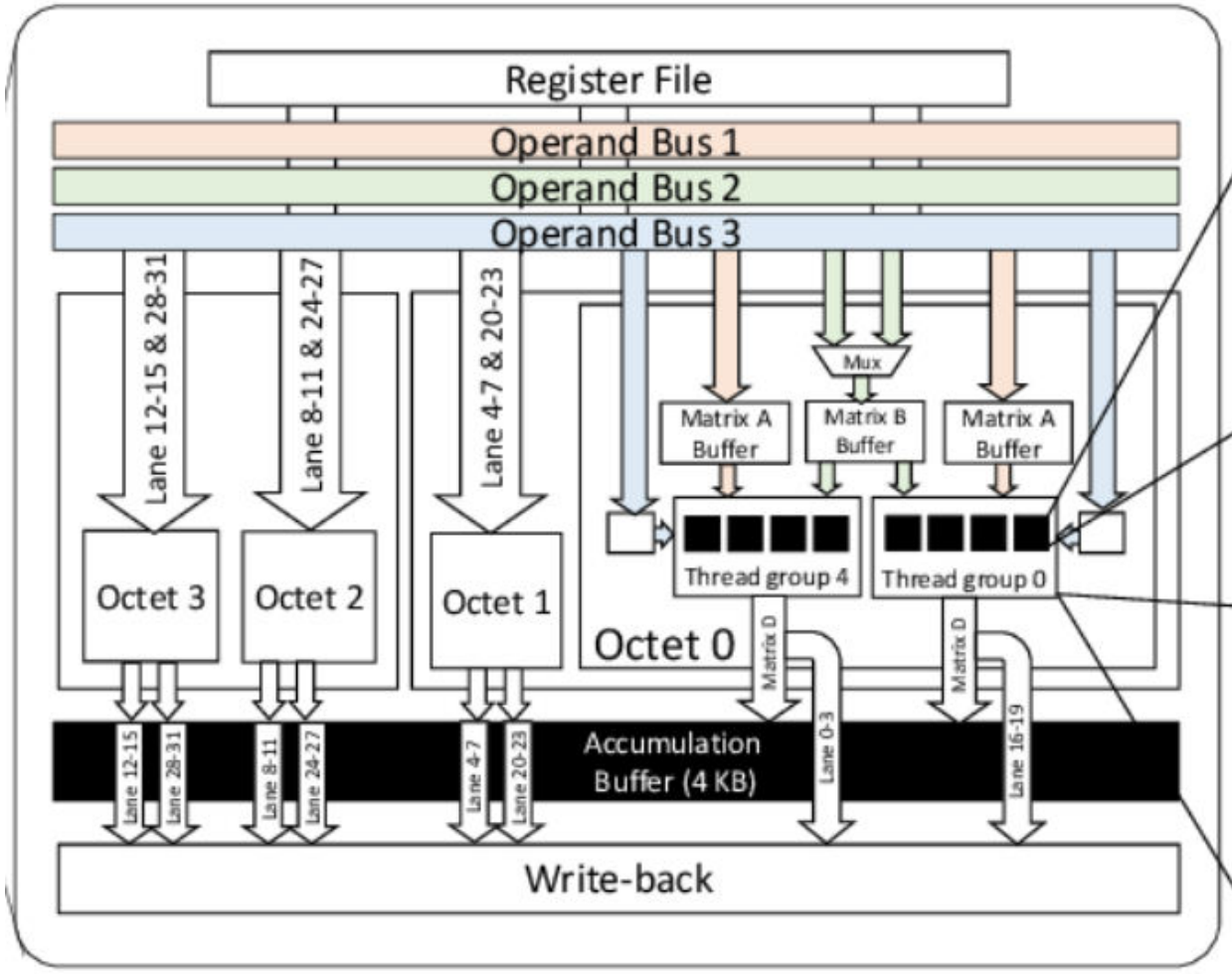
(a) Disassembled SASS instructions for Mixed precision mode

		Cumulative Clock Cycles
SET1	HMMA.884.F16.F16.STEP0 R4, R22.reuse.T, R12.reuse.T, R4;	12
	HMMA.884.F16.F16.STEP1 R6, R22.T, R12.T, R6;	21
SET2	HMMA.884.F16.F16.STEP0 R4, R16.reuse.T, R14.reuse.T, R4;	25
	HMMA.884.F16.F16.STEP1 R6, R16.T, R14.T, R6;	34
SET3	HMMA.884.F16.F16.STEP0 R4, R18.reuse.T, R8.reuse.T, R4;	38
	HMMA.884.F16.F16.STEP1 R6, R18.T, R8.T, R6;	47
SET4	HMMA.884.F16.F16.STEP0 R4, R2.reuse.T, R10.reuse.T, R4;	51
	HMMA.884.F16.F16.STEP1 R6, R2.T, R10.T, R6;	64

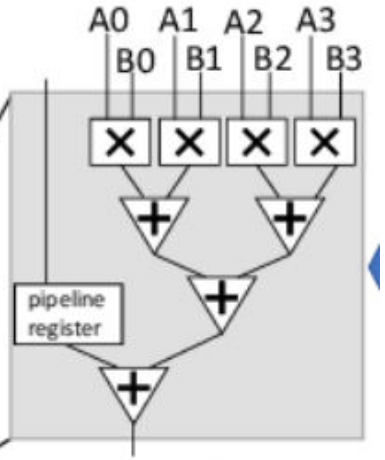
(b) Disassembled SASS instructions for FP16 mode

Each HMMA is executed by all 8 thread groups

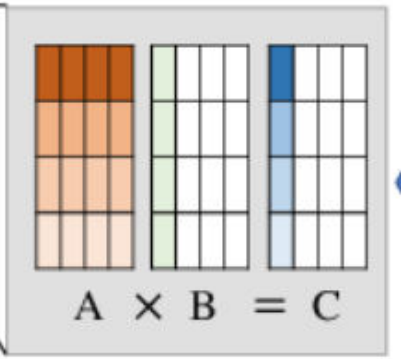
Tensor Core uArch



(b) Tensor Core Architecture



(c) Four-element Dot Product (FED

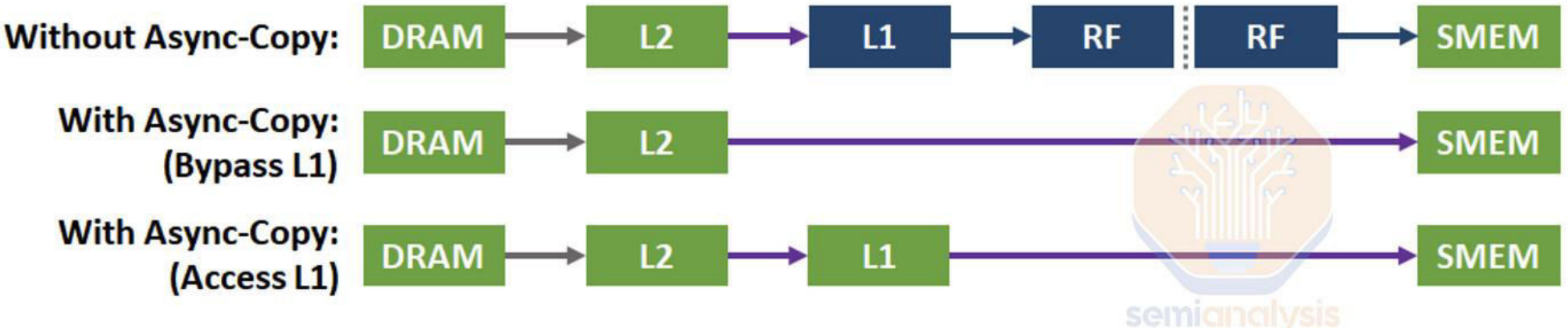


(e) Inner product in thread-group

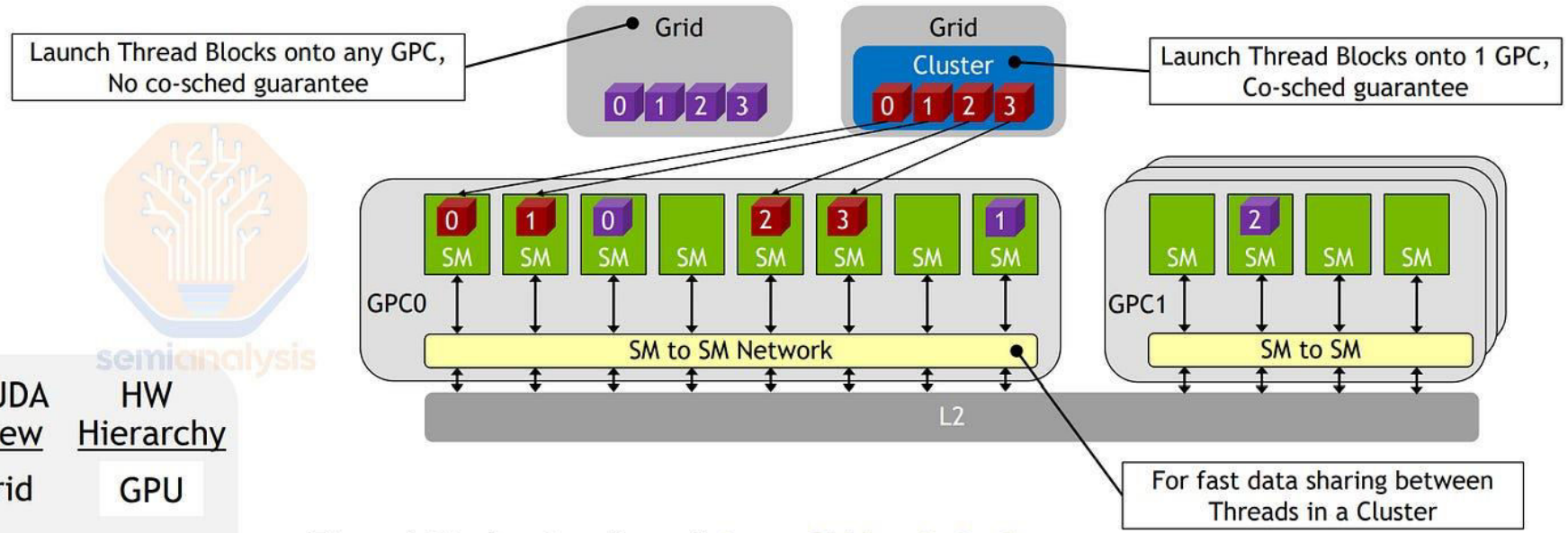
Tensor Core Evolution: Asynchronous Copy

thread-level async copy: cp.async

https://docs.nvidia.com/cuda/parallel-thread-execution/index.html?highlight=tcgen05%2520cp#data-movement-and-conversion-instructions-non-bulk-copy



Thread Block Cluster (TBC) and Graphics Processing Cluster (GPC)



CUDA View	HW Hierarchy
Grid	GPU
Cluster	GPC
Thread Block	SM
Threads	Lanes

Thread Blocks distributed 1 per SM by default

Cluster size: Up to 16 Thread Blocks

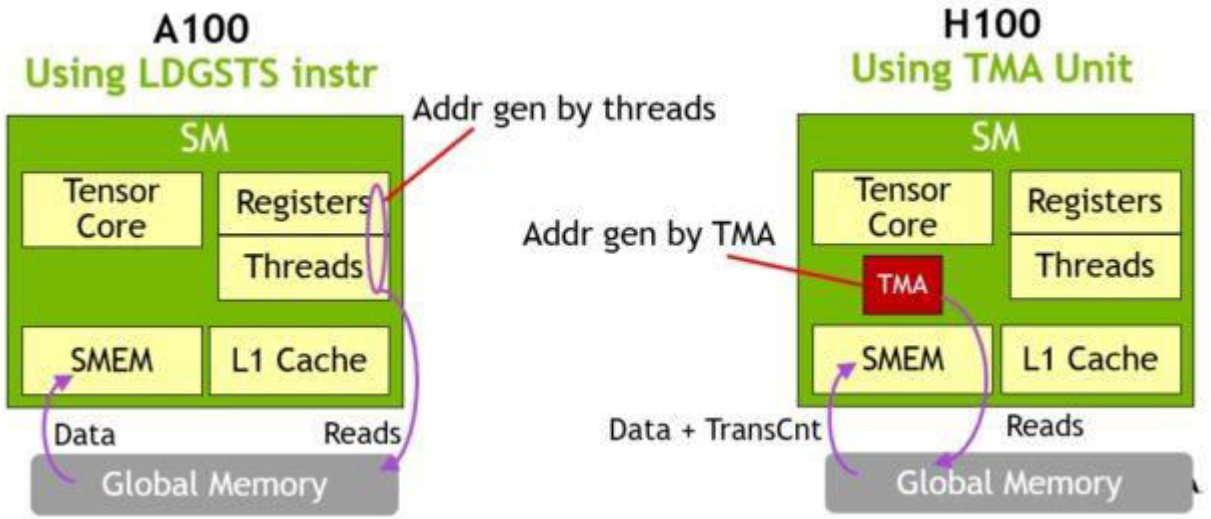
CUDA exposes Clusters through the Cooperative Group API

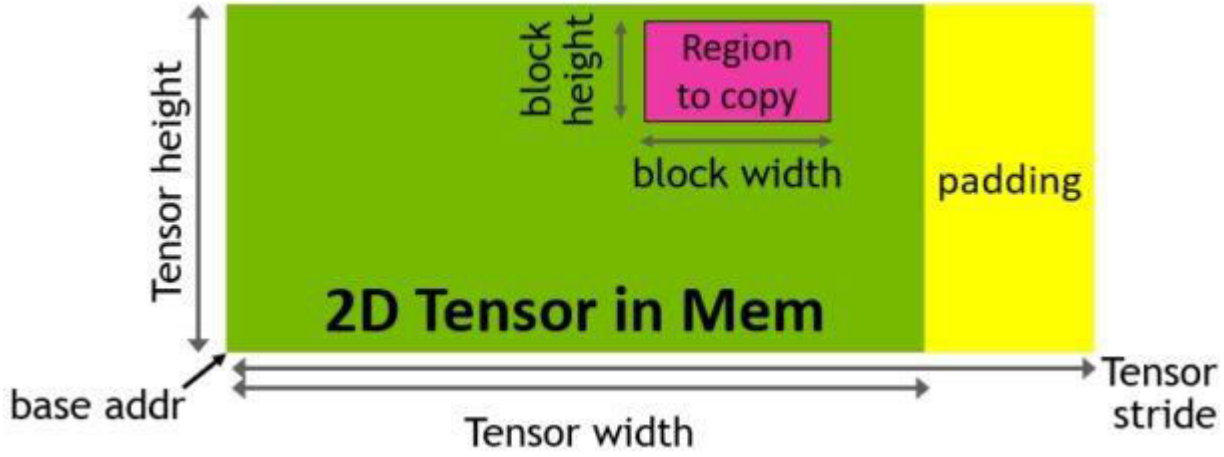
See also *CUDA: New Features and Beyond* (S41486, Tue 11am PT)

Tensor Memory Accelerator

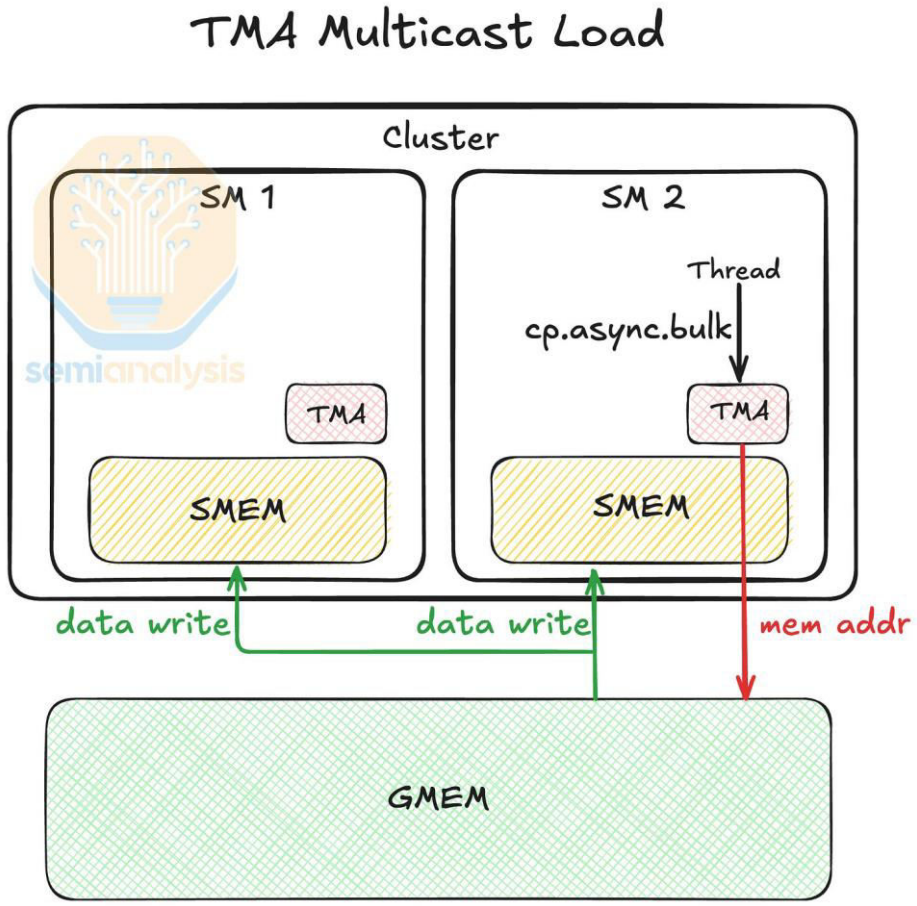


Hopper arch.





Tensor Memory Accelerator (cont.)

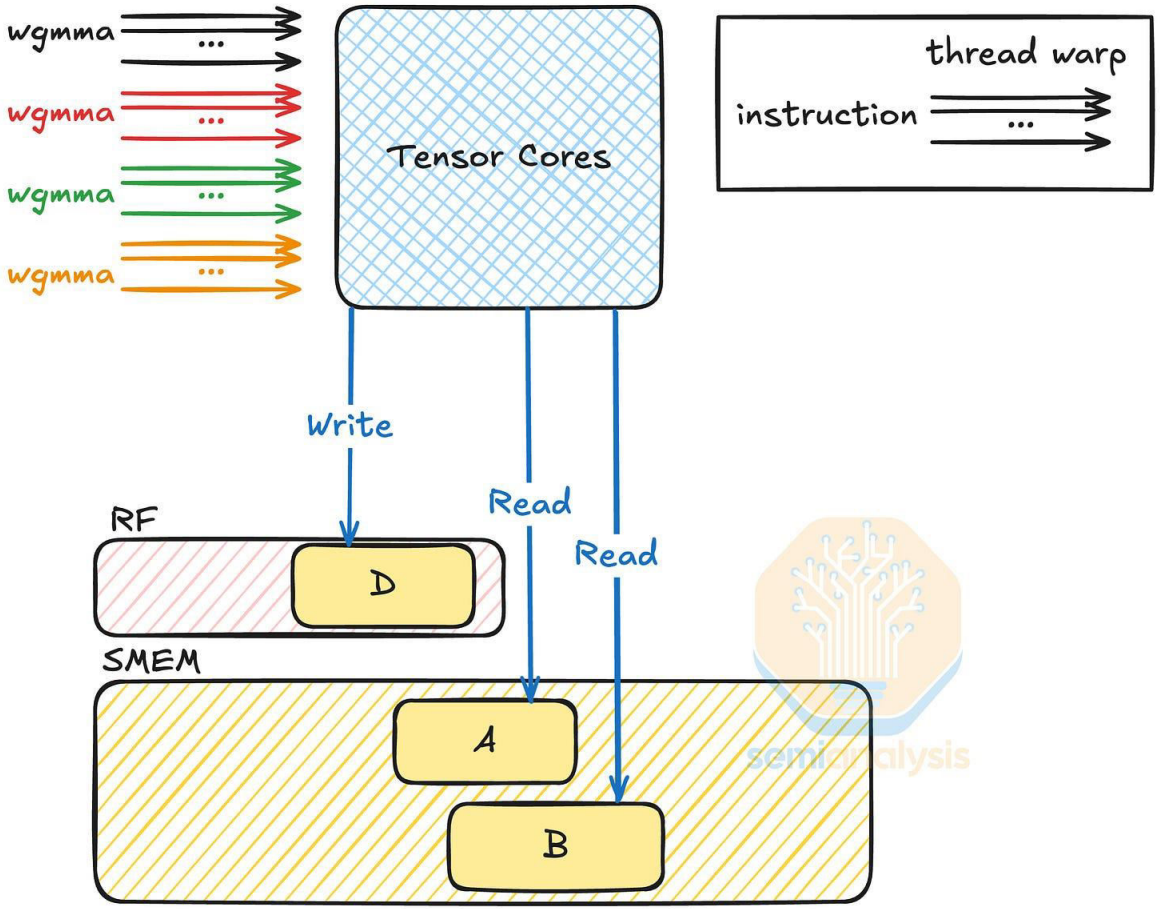


HW accelerated reduction operation:
cp.reduce.async.bulk

https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#data-movement-and-conversion-instructions-cp-reduce-async-bulk

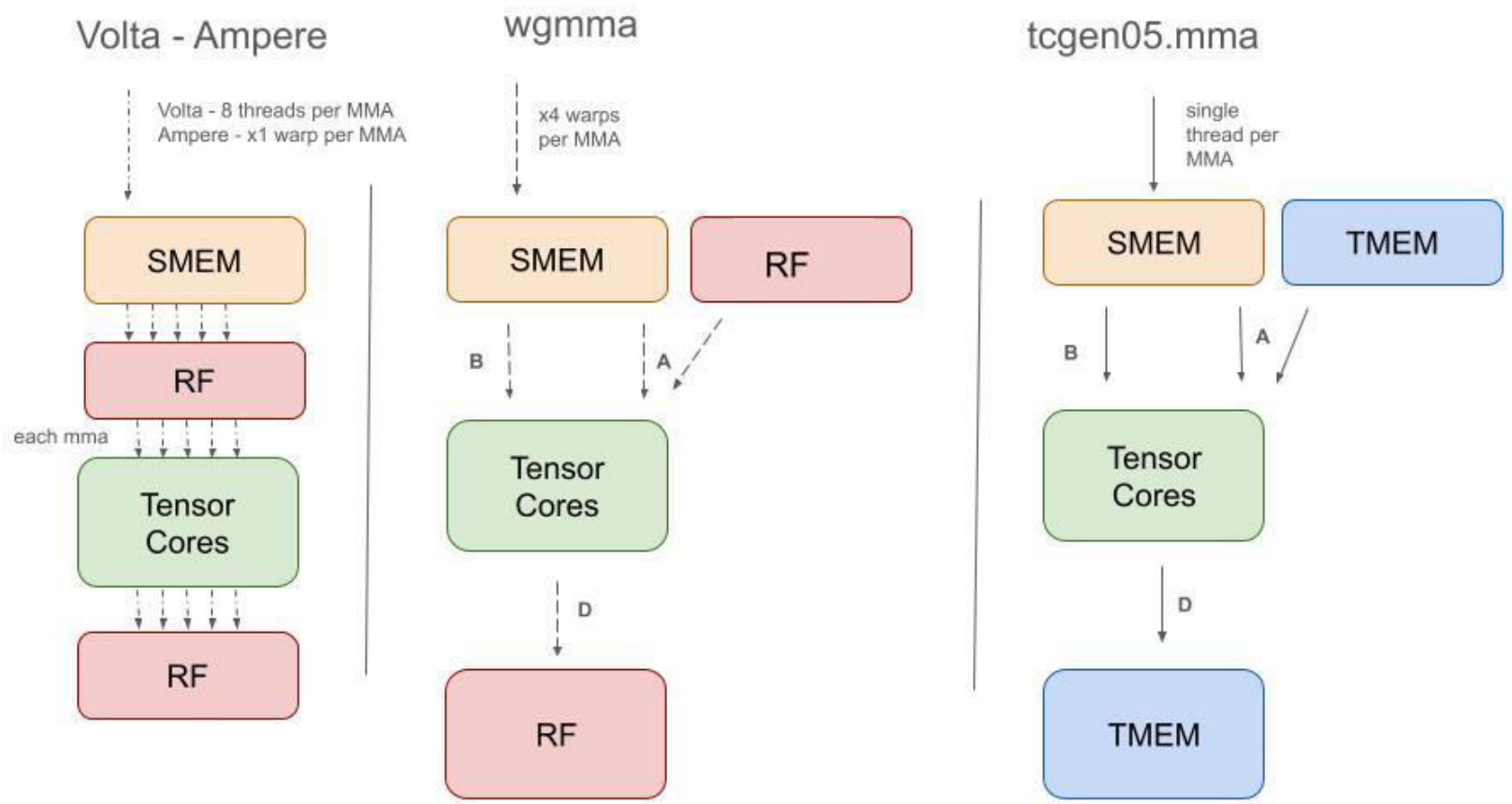
Source: https://newsletter.semianalysis.com/p/nvidia-tensor-core-evolution-from-volta-to-blackwell

Warpgroup-level Asynchronous MMA (wgmma)



Source: https://newsletter.semianalysis.com/p/nvidia-tensor-core-evolution-from-volta-to-blackwell

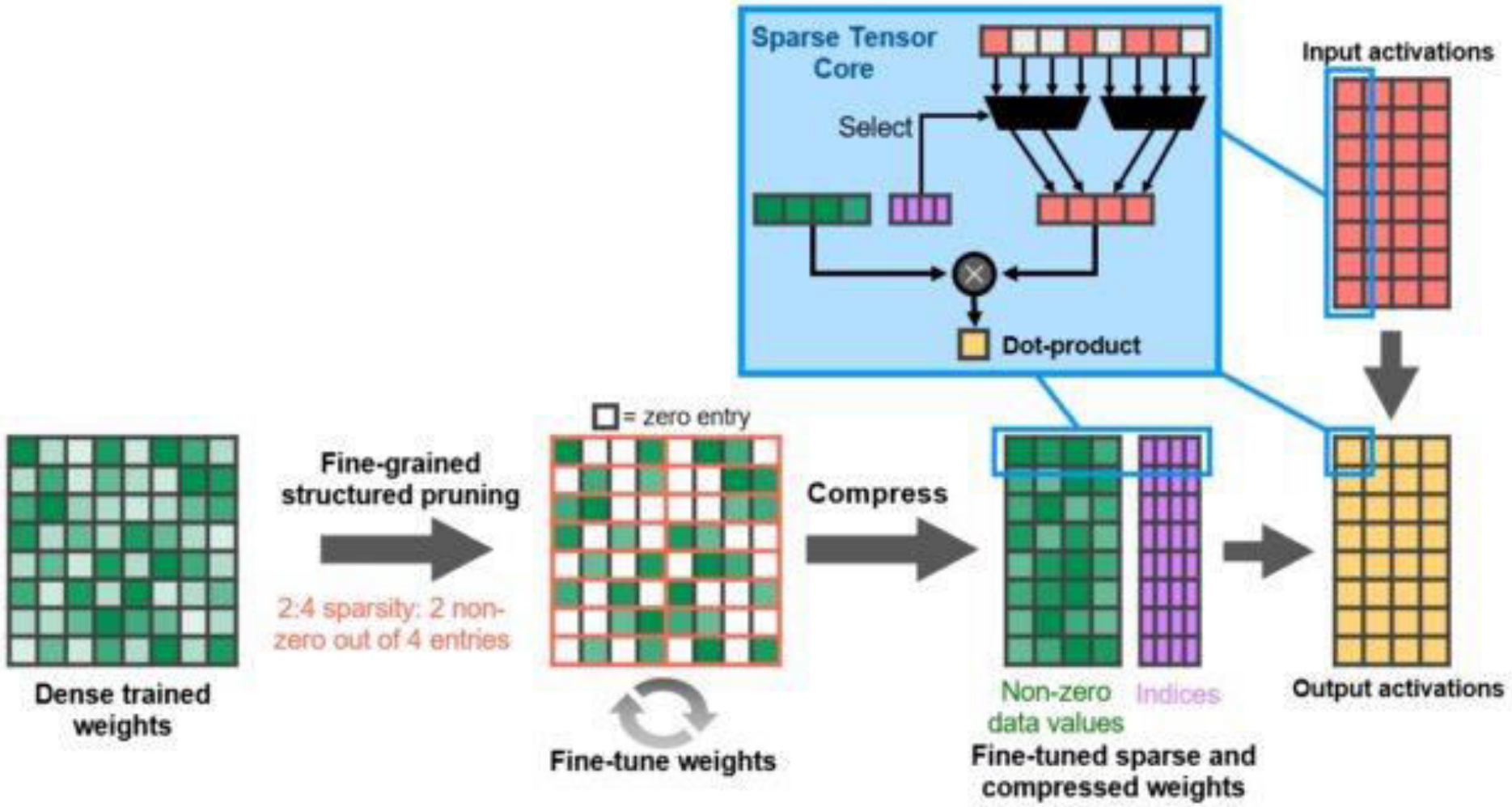
Tensor Memory (tcgen05 instructions)



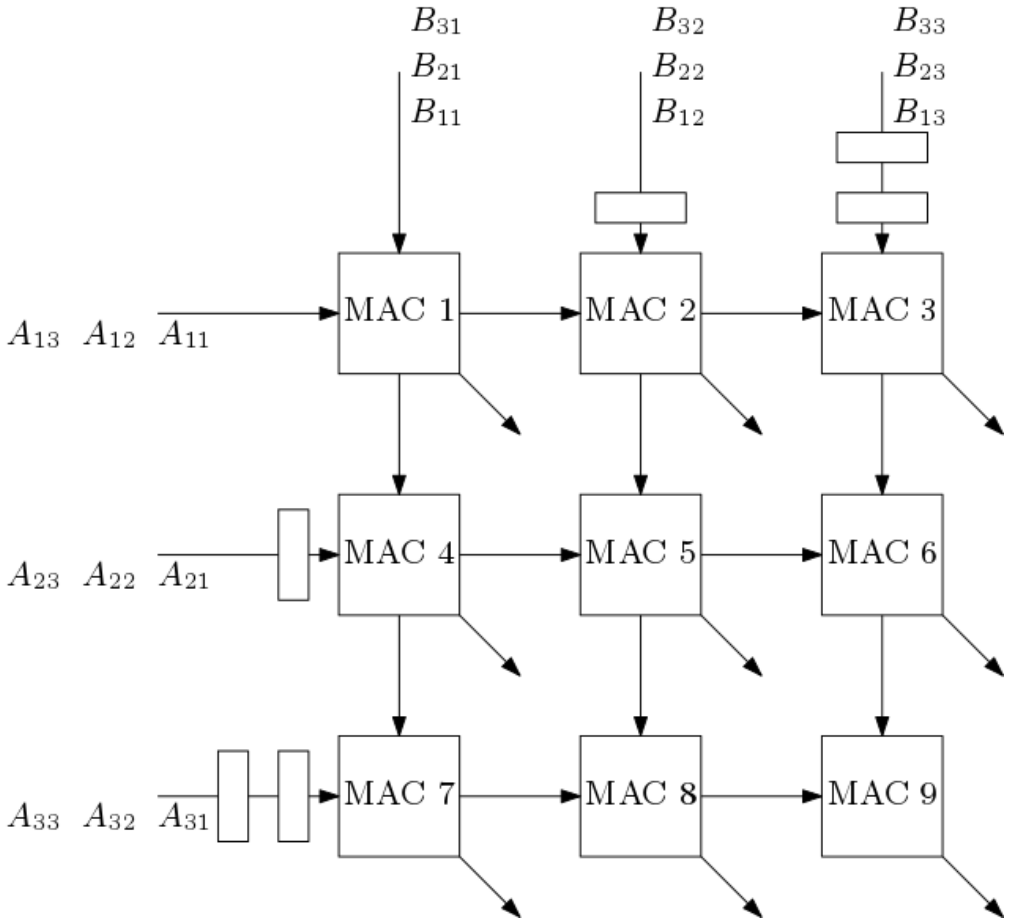
Hopper

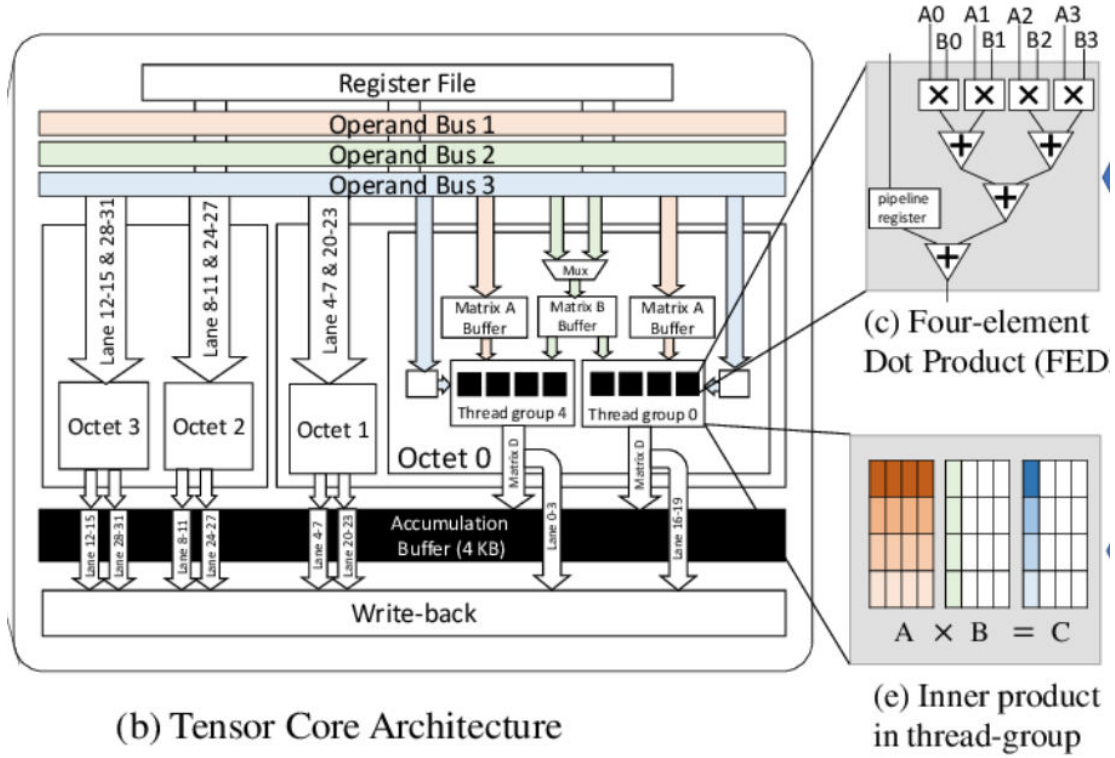
Blackwell

Improvements: more flexible matrix shapes, precisions, sparse matrices



Tensor Core vs. Systolic Array





(b) Tensor Core Architecture

(c) Four-element Dot Product (FED

(e) Inner product in thread-group

Informational videos

- https://www.youtube.com/watch?v=Cak8ASX7NOk
- https://www.youtube.com/watch?v=KniNW8Xe4Us
- https://www.youtube.com/watch?v=hQ9GPnV0-50

Resources

- https://blog.paperspace.com/understanding-tensor-cores/
- https://developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth/
- https://www.anandtech.com/show/12673/titan-v-deep-learning-deep-dive/3
- Raihan et. al., Modeling Deep Learning Accelerator Enabled GPUs, arXiv:1811.08309v2, 2019
- Sun et.al., Dissecting Tensor Cores via Microbenchmarks, TPDS, 2023